SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
 <input type="text" name="name">
 <input type="email" name="email">
 <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

Answer:

Given HTML Structure

```
<form action="/register" method="POST">
    <input type="text" name="name">
    <input type="email" name="email">
    <button>Submit</button>
</form>
```

1. Appropriate HTML5 Semantic Elements Missing

The given structure contains only a <form> element. The following semantic elements can improve the structure:

- <header> – Represents the heading or introductory content of the symposium portal.
- <main> – Represents the main content of the webpage.
- <section> – Groups related content, such as registration details.
- <footer> – Contains footer information.
- <label> – Provides accessible names for form fields.
- <fieldset> and <legend> – Group related form controls and describe their purpose.

Using semantic elements improves readability, accessibility, SEO, and maintainability.

2. Standards-Compliant HTML5 Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Symposium Registration</title>
</head>

<body>

    <header>
        <h1>Online Symposium 2026</h1>
```

```
    <p>Register to participate in our college symposium.</p>
</header>

<main>
    <section aria-labelledby="registration-heading">
        <h2 id="registration-heading">Student Registration</h2>

        <form action="/register" method="POST">

            <fieldset>
                <legend>Personal Information</legend>

                <label for="name">Full Name:</label>
                <input
                    type="text"
                    id="name"
                    name="name"
                    required
                    autocomplete="name"
                >

                <br><br>

                <label for="email">Email Address:</label>
                <input
                    type="email"
                    id="email"
                    name="email"
                    required
                    autocomplete="email"
                >
```

```
            <br><br>

            <button type="submit">Submit Registration</button>
        </fieldset>

    </form>
  </section>
</main>

<footer>
  <p>&copy; 2026 College Symposium</p>
</footer>

</body>
</html>
```

This structure is standards-compliant because it includes:

- HTML5 document declaration
- Language declaration
- Character encoding
- Responsive viewport
- Semantic HTML5 elements
- Proper labels and form controls
- Explicit submit button type
- Required validation attributes

3. Interpretation of the HTTP Request

When the student submits the form, the browser creates an HTTP POST request.

For example:

```
POST /register HTTP/1.1
Host: www.example.com
```

Content-Type: application/x-www-form-urlencoded

Content-Length: ...

The form data is placed in the request body:

name=Gnaneswari&email=student@example.com

The request process is:

1. The user enters the name and email.
2. The browser validates the form fields.
3. The browser sends a POST request to /register.
4. The form data is included in the request body.
5. The server receives and processes the data.
6. The server sends an HTTP response to the browser.

The POST method is suitable for registration because the data is submitted to the server for processing and is not displayed directly in the URL.

4. HTTP Response Cycle from Server to Browser

The HTTP response cycle occurs as follows:

Step 1: Browser Sends Request

POST /register HTTP/1.1

The browser sends the form data to the server.

Step 2: Server Processes the Request

The server may:

- Validate the name and email.
- Store the registration in a database.
- Check whether the student is already registered.

Step 3: Server Sends Response

A successful response may be:

HTTP/1.1 200 OK

Content-Type: text/html

The server may return:

<h1>Registration Successful</h1>

Thank you for registering for the symposium.

Alternatively, the server may redirect the browser:

HTTP/1.1 302 Found

Location: /success.html

Step 4: Browser Displays the Result

The browser receives the response and either:

- Displays the returned HTML page, or
- Follows the redirect and loads another page.

Thus, the HTTP communication follows the cycle:

Client → HTTP Request → Server → Processing → HTTP Response → Browser

5. Improvements for Usability and Accessibility

The following improvements can be implemented:

1. Use labels for all input fields so screen readers can identify them.
2. Use required to prevent empty form submission.
3. Use type="email" to provide automatic email validation.
4. Use autocomplete attributes to make form completion easier.
5. Provide meaningful error messages.
6. Use sufficient color contrast.
7. Ensure the form can be operated using only the keyboard.
8. Use aria-describedby when additional instructions are provided.
9. Use responsive design for mobile devices.
10. Provide a clear success or failure message after submission.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
 <title>Company</title>
 </head>
 <body>
 <div>
 <h1>Welcome
 <p>About us
 </div>
 </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

Answer:

1. Improper Nesting and Missing Closing Tags

The given HTML contains several structural errors:

Missing <!DOCTYPE html>

The document does not declare HTML5:

<!DOCTYPE html>

Without the doctype, browsers may enter **quirks mode**, causing inconsistent rendering.

Missing Closing </h1> Tag

The following element is not closed:

<h1>Welcome

It should be:

<h1>Welcome</h1>

Missing Closing Tag

The paragraph is also not closed:

About us

It should be:

About us

Missing Semantic Structure

The page does not contain semantic elements such as:

- <header>
- <main>
- <section>
- <footer>

The use of a generic does not clearly describe the purpose of the content.

2. How Different Browsers May Interpret the Structure

Browsers use HTML parsing algorithms to handle invalid markup. However, different browsers or rendering modes may produce differences in:

- DOM tree construction
- Margin and spacing
- Element nesting
- Text positioning
- CSS inheritance
- Layout rendering

For example, one browser may automatically close the <h1> element before the <p> element, while another browser may construct the DOM differently.

If the document lacks:

<!DOCTYPE html>

the browser may use **quirks mode**. Quirks mode attempts to imitate older browser behavior and can result in:

- Different box model calculations
- Inconsistent element sizes
- Different spacing
- Layout differences across browsers

Therefore, valid HTML is important for predictable rendering.

3. Missing Semantic Elements

The following semantic elements can be added:

<header>

Used for the main heading or introductory content.

<main>

Represents the main content of the page.

<section>

Groups related content.

<footer>

Contains copyright or contact information.

A semantic structure is more meaningful than using only elements.

4. Corrected W3C-Compliant HTML Structure

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>

<body>

   <header>
      <h1>Welcome to Our Company</h1>
   </header>

   <main>
      <section>
         <h2>About Us</h2>
         <p>We provide innovative technology solutions to our customers.</p>
      </section>
   </main>

   <footer>
      <p>&copy; 2026 Our Company. All rights reserved.</p>
   </footer>

</body>
```

</html>

This structure is valid because:

- The HTML5 doctype is declared.
- All elements are properly nested.
- All required closing tags are included.
- Semantic elements are used.
- The page includes language and character encoding declarations.
- The viewport is configured for responsive design.

5. Techniques to Ensure Cross-Browser Compatibility

The following techniques should be used:

1. Use valid HTML5 and CSS according to web standards.
2. Always include:

   <!DOCTYPE html>
3. Use proper opening and closing tags.
4. Test the website in major browsers such as Chrome, Firefox, Edge, and Safari.
5. Use CSS resets or normalization techniques.
6. Avoid browser-specific or obsolete features.
7. Use feature detection instead of assuming browser support.
8. Use responsive design techniques.
9. Use browser developer tools to identify layout problems.
10. Validate HTML and CSS using standards-compliance validators.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.

5. Suggest improvements for **secure data transmission**.

Answer:

1. Issues in the Form Configuration

The main issue is:

<button type="button">Login</button>

A button with:

type="button"

is a normal button. It does **not automatically submit the form**.

Therefore, clicking the button does not trigger the normal form submission process.

The form also uses:

method="GET"

for a password field. This is insecure because GET data is included in the URL.

2. Why the HTTP Request Is Not Triggered

The browser submits a form automatically when:

<button type="submit">

or:

<input type="submit">

is used.

However, the given code contains:

<button type="button">Login</button>

This button only performs a general button action. It does not submit the form.

Therefore:

1. The user enters username and password.
2. The user clicks Login.
3. The button does not submit the form.
4. No normal HTTP request is generated.

To trigger form submission, the button should be:

<button type="submit">Login</button>

3. Role of GET Method in This Scenario

The GET method sends form data as query parameters in the URL.

For example:

/submit?username=Gnaneswari&password=12345

The advantages of GET include:

- Data can be bookmarked.
- It is suitable for retrieving information.
- It is commonly used for search forms.

However, GET is not suitable for passwords because:

- Data appears in the browser address bar.
- Data may be stored in browser history.
- Data may appear in server logs.
- Data may be exposed through proxy logs.

Therefore, confidential information such as passwords should not be sent using GET.

4. Corrected Implementation

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
</head>

<body>

    <main>
        <h1>Login</h1>

        <form action="/submit" method="POST">
```

```
        <label for="username">Username:</label>
        <input
          type="text"
          id="username"
          name="username"
          required
          autocomplete="username"
        >

        <br><br>

        <label for="password">Password:</label>
        <input
          type="password"
          id="password"
          name="password"
          required
          autocomplete="current-password"
        >

        <br><br>

        <button type="submit">Login</button>

      </form>
    </main>

</body>

</html>
```

The corrected implementation:

- Uses POST instead of GET.
- Uses type="submit".
- Adds labels.
- Adds required validation.
- Uses autocomplete attributes.
- Uses semantic HTML5 structure.

5. Improvements for Secure Data Transmission

The following security improvements should be implemented:

1. Use HTTPS

The application should use HTTPS instead of HTTP.

https://example.com/submit

HTTPS encrypts data during transmission.

2. Use POST for Passwords

Passwords should be submitted using:

<form method="POST">

This prevents them from appearing in the URL.

3. Use Server-Side Validation

The server must validate and sanitize all input data because client-side validation can be bypassed.

4. Hash Passwords

Passwords should never be stored as plain text. The server should store securely hashed passwords.

5. Use CSRF Protection

A CSRF token should be used to prevent unauthorized form submissions.

6. Use Secure Cookies

Authentication cookies should use security attributes such as:

- Secure
- HttpOnly
- SameSite

7. Avoid Logging Sensitive Data

Passwords and confidential information should never be stored in application logs or URLs.

CONCLUSION

HTML5 semantic elements, proper nesting, valid syntax, and correct HTTP methods are essential for developing reliable web applications. Semantic HTML improves accessibility and maintainability, while valid markup helps achieve consistent cross-browser behavior. Correct form configuration is also important because the button type determines whether the form submission process occurs. Finally, sensitive information such as passwords must be transmitted using secure methods such as HTTPS and POST.

These practices help developers create standards-compliant, accessible, compatible, and secure web applications.

Code of Conduct:

I T Gnaneswari(192372183) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:T Gnaneswari

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions

Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand